Trabalho de IA — 2º Bimestre

Implementação de uma Rede Neural MLP (Multilayer Perceptron) com o algoritmo Backpropagation

1 Visão geral

Neste trabalho, cada equipe vai construir **do zero** uma ferramenta capaz de treinar e avaliar uma rede neural do tipo **Perceptron Multicamadas (MLP)** usando o algoritmo **Backpropagation**. A ferramenta deve ler uma base de dados em CSV, preparar os dados, permitir a configuração da rede e do treinamento, acompanhar o aprendizado em tempo real e, ao final, medir o desempenho da rede em dados que ela nunca viu, por meio de uma **matriz de confusão**.

A base de referência disponibilizada possui **6 atributos de entrada** e **5 classes de saída**, mas a ferramenta deve funcionar com **qualquer base** que siga o mesmo formato.

Objetivos de aprendizagem

- Compreender e implementar a propagação direta (forward) e a retropropagação do erro (backward) em uma MLP.
- Relacionar a escolha da função de transferência com sua derivada e com o cálculo dos gradientes.
- Aplicar boas práticas de preparação de dados: normalização, separação treino/teste e codificação da saída.
- Interpretar a curva de erro, identificar estagnação (platô) e agir sobre a taxa de aprendizagem.
- Avaliar um classificador por meio da matriz de confusão e da acurácia.

2 Regras de desenvolvimento

Linguagem livre — algoritmo autoral

- **A linguagem de programação é de livre escolha** da equipe (Python, Java, C#, C/C++, JavaScript/TypeScript, Kotlin, Go, Rust, etc.), assim como o tipo de aplicação: desktop, web ou mobile.
- É **proibido** usar bibliotecas ou frameworks de IA/Aprendizado de Máquina, como **TensorFlow, Keras, PyTorch, scikit-learn, JAX, ML.NET, Weka, Brain.js, Synaptic** ou equivalentes, inclusive apenas para calcular gradientes, ativações ou a matriz de confusão.
- Toda a lógica da rede — inicialização dos pesos, propagação, cálculo dos erros/gradientes, atualização dos pesos, critérios de parada, detecção de platô e avaliação — deve ser **implementada pela equipe**.
- **São permitidas** bibliotecas de apoio que não implementem IA: leitura de arquivos/CSV, interface gráfica, gráficos (ex.: Chart.js, Matplotlib, JavaFX Charts) e operações numéricas elementares com vetores e matrizes.
- Todos os integrantes devem ser capazes de **explicar qualquer trecho do código** na apresentação. Códigos que a equipe não consiga explicar serão desconsiderados.

3 Base de dados

Serão disponibilizados dois arquivos: um para **treinamento** e outro para **teste**. Ambos seguem o padrão **CSV separado por vírgula**, com uma linha de cabeçalho. **A última coluna é sempre a classe**; as demais são os atributos numéricos de entrada.

```
X1,X2,X3,X4,X5,X6,classe
1,19,35,28,17,4,CA
4,22,38,32,14,9,CB
-2,16,39,29,21,5,CB
-2,22,33,32,22,1,CE
...
```

A ferramenta deve **identificar automaticamente** o número de atributos (neurônios de entrada) e o número de classes distintas (neurônios de saída). Atenção a detalhes do arquivo, como espaços antes do rótulo da classe (ex.: `" CA"`), que devem ser tratados.

Codificação da saída (one-hot): a rede usa **um neurônio de saída por classe**. Para um exemplo da classe CB, a saída desejada é `0 1 0 0 0`. Na classificação, a classe atribuída pela rede é a do neurônio com **maior saída**.

4 Requisitos funcionais

Todos os requisitos abaixo são **obrigatórios**. A ordem sugere um caminho de desenvolvimento.

ID	Requisito	O que deve ser feito
RF01	**Carregar dados**	Ler arquivos CSV com cabeçalho; detectar nº de atributos e de classes; exibir os dados carregados em uma tabela; aceitar outras bases no mesmo formato.
RF02	**Definir treino e teste**	Permitir duas opções: (a) carregar dois arquivos separados (treino e teste) ou (b) carregar um único arquivo e dividi-lo informando o percentual de treino (ex.: 70% / 30%), com embaralhamento dos exemplos.
RF03	**Normalizar os dados**	Normalizar as entradas entre 0 e 1 (min-max). Os valores mínimo e máximo de cada atributo devem ser calculados somente no treino e guardados para normalizar o teste. Tratar o caso máximo = mínimo.
RF04	**Configurar a arquitetura**	Camada de entrada = nº de atributos; camada de saída = nº de classes. Camada oculta calculada pela média aritmética ou geométrica (seção 5.2), com opção de valor manual.
RF05	**Escolher a função de transferência**	Linear, Logística ou Tangente Hiperbólica, com as respectivas derivadas (seção 5.3).
RF06	**Definir a taxa de aprendizagem (η)**	Aceitar somente valores $0 < \eta \leq 1$, com validação e mensagem de erro amigável.
RF07	**Critérios de parada**	Por nº de épocas e por erro mínimo (limiar). O treinamento termina quando o primeiro dos dois critérios for atingido. Exibir qual critério encerrou o treino.
RF08	**Treinar com Backpropagation**	Pesos iniciais aleatórios pequenos; atualização após cada exemplo apresentado (padrão a padrão); cálculo do erro médio da rede em cada época.
RF09	**Acompanhar o treinamento**	Exibir em tempo real a época atual, o erro médio da época, o η atual e o gráfico da curva de erro por época. Permitir pausar/interromper.

ID	Requisito	O que deve ser feito
RF10	**Visualizar a arquitetura**	Desenhar a rede configurada (neurônios e conexões). Ao passar o mouse sobre uma conexão, mostrar o valor do peso.
RF11	**Detectar platô**	Identificar estagnação do erro médio e oferecer as opções descritas na seção 6.
RF12	**Testar a rede**	Aplicar a rede treinada à base de teste e exibir a matriz de confusão (seção 7) e a acurácia geral.

Extras (opcionais, valem bônus)

- Salvar e carregar os pesos de uma rede treinada.
- Exibir precisão e revocação por classe, além da acurácia.
- Exportar o histórico do erro e a matriz de confusão (CSV ou imagem).
- Permitir fixar a semente aleatória (seed) para repetir um experimento.
- Comparar, em um mesmo gráfico, o treino com diferentes funções de transferência ou taxas de aprendizagem.

5 Fundamentação para a implementação

5.1 Normalização (min-max)

$$ValorNormalizado = (ValorAtributo - MenorValorAtrib) / (MaiorValorAtrib - MenorValorAtrib)$$

Os valores de menor e maior de cada atributo vêm **da base de treino** e são reutilizados, sem recálculo, na base de teste.

5.2 Número de neurônios na camada oculta

O número de neurônios ocultos costuma ser definido **empiricamente**. Neurônios demais levam a rede a **memorizar** o treino (*overfitting*), em vez de extrair características gerais que permitam a generalização; neurônios de menos obrigam a rede a gastar tempo excessivo buscando uma representação adequada. Recomenda-se manter o número baixo, mas não tão baixo quanto o estritamente necessário. Duas heurísticas comuns:

$$Média\ aritmética = (Nº\ Entradas + Nº\ Saídas) / 2$$

$$Média\ geométrica = \sqrt{(Nº\ Entradas \times Nº\ Saídas)}$$

Para a base de referência: aritmética = (6 + 5) / 2 = 5,5 e geométrica = √30 ≈ 5,48. Arredonde para um inteiro e informe na interface o critério utilizado.

5.3 Funções de transferência e derivadas

Nas fórmulas, **i** é a saída do neurônio, isto é, i = f(net), tanto na camada oculta quanto na de saída.

Função	f(net)	f'(net)	Faixa de saída
Linear	net / 10	1 / 10	(−∞, +∞)
Logística	1 / (1 + e^(−net))	i · (1 − i)	(0, 1)
Tangente hiperbólica	(1 − e^(−2·net)) / (1 + e^(−2·net))	1 − i²	(−1, 1)

5.4 Gradientes do erro e atualização dos pesos

Para cada exemplo apresentado: (1) propague as entradas até a saída; (2) calcule o erro de cada neurônio de saída; (3) retropropague o erro para a camada oculta; (4) atualize os pesos.

Erro (gradiente) de um neurônio G da camada de saída:

$$Erro_G = (Desejado_G - i_G) \cdot f'(net_G)$$

Erro de um neurônio C da camada oculta (soma sobre todos os neurônios G de saída ligados a ele):

$$Erro_C = (\Sigma_G \ Erro_G \cdot w_G,C) \cdot f'(net_C)$$

Atualização de um peso que liga o neurônio de origem (saída i) ao neurônio de destino:

$$w_novo = w_atual + \eta \cdot Erro_destino \cdot i_origem$$

Exemplos com a função logística: `Erro_G = (Desejado − i_G) · i_G · (1 − i_G)` e `Erro_C = (Σ Erro_G · w_G,C) · i_C · (1 − i_C)`. Com a tangente hiperbólica, troque o termo da derivada por `(1 − i²)`.

Erro da rede: para cada exemplo, calcule o erro quadrático `E = ½ · Σ (Desejado − i)²` sobre os neurônios de saída; o **erro médio da época** é a média de E sobre todos os exemplos de treino. É esse valor que alimenta o critério de parada por erro, o gráfico e a detecção de platô.

6 Detecção de platô

Durante o treinamento, a ferramenta deve verificar se o erro médio por época **estabilizou** (platô), situação em que a rede continua treinando sem melhorar.

1. Mantenha uma janela com o erro médio das **últimas 10 épocas** (no mínimo).
2. Calcule o **desvio padrão** desses valores. Se estiver entre 0 e um limiar (ex.: 0,00001), considere o treinamento estagnado.
3. Pause e ofereça ao usuário três opções: **interromper** o treinamento, **continuar sem alterar η**, ou **continuar reduzindo η em 10%** (η_novo = 0,9 · η).
4. Se o platô persistir, apresente as opções novamente. Registre cada ocorrência e cada redução de η (ex.: em um log na tela).

7 Matriz de confusão

A matriz de confusão mostra quantos exemplos da base de **teste** foram classificados corretamente — na **diagonal principal** — e como os erros se distribuem. As **linhas** indicam a classe correta e as **colunas**, a resposta da rede. Exemplo com três classes e 15 exemplos de teste por classe:

Real \ Prevista	Mamíferos	Aves	Peixes
Mamíferos	**13**	2	0
Aves	2	**10**	3
Peixes	0	3	**12**

Leitura: dos 15 mamíferos apresentados, 13 foram classificados corretamente e 2 foram confundidos com aves. A acurácia geral é a soma da diagonal dividida pelo total: (13 + 10 + 12) / 45 ≈ 77,8%.

8 Esboço de interface (sugestão)

As telas a seguir são **apenas uma sugestão** de organização. A equipe tem liberdade para propor o próprio layout, desde que todos os requisitos estejam presentes e acessíveis. Uma organização em etapas — **Dados → Treinamento → Avaliação** — costuma deixar o fluxo mais claro.

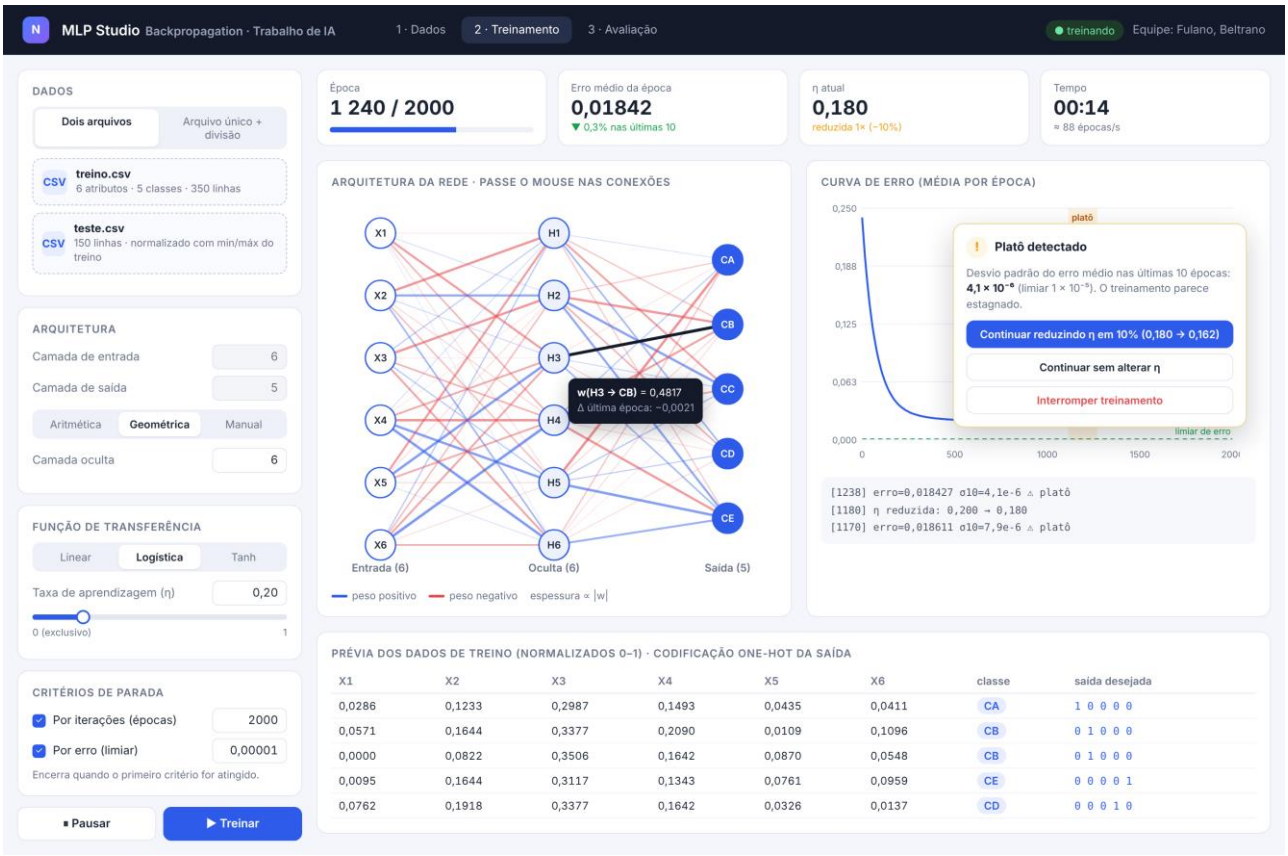


Figura 1 — Tela de treinamento: configuração, arquitetura com pesos no hover, curva de erro e aviso de platô.

Elementos da Figura 1:

- **Painel lateral:** escolha do modo de dados (dois arquivos ou arquivo único com divisão), camadas de entrada e saída preenchidas automaticamente, cálculo da camada oculta (aritmética, geométrica ou manual), função de transferência, η e critérios de parada.
- **Indicadores:** época atual, erro médio da época, η atual (com histórico de reduções) e tempo de treino.
- **Arquitetura:** neurônios e conexões desenhados; cor indica o sinal do peso e espessura, a magnitude; o valor aparece ao passar o mouse.
- **Curva de erro:** erro médio por época, linha do limiar de erro e destaque da região de platô, com a caixa de decisão do usuário e um log de eventos.
- **Prévia dos dados:** valores normalizados e a saída desejada em codificação one-hot.

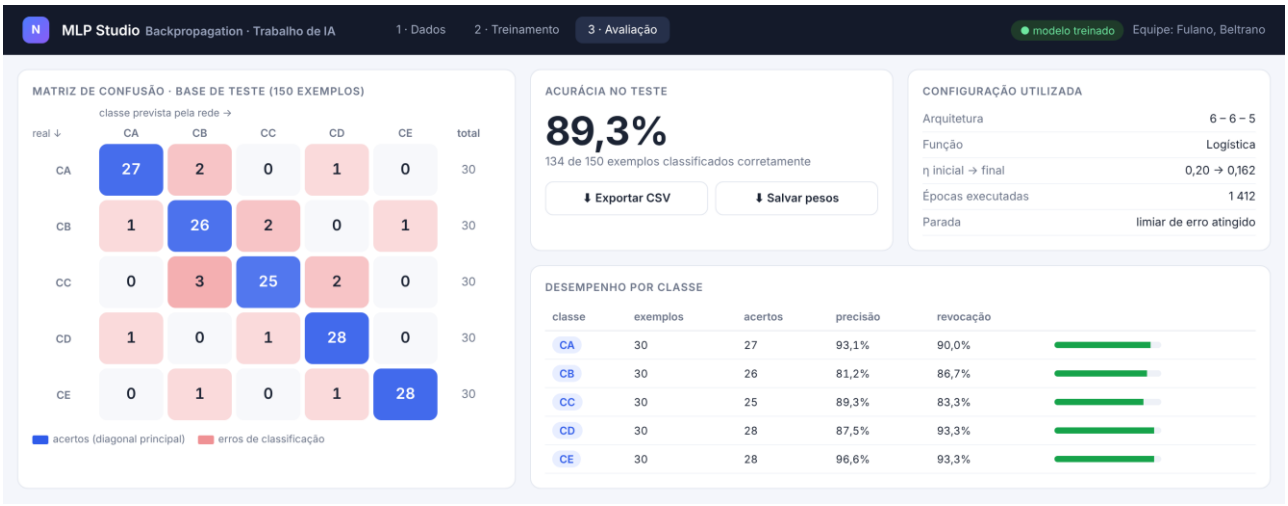


Figura 2 — Tela de avaliação: matriz de confusão, acurácia, configuração usada e desempenho por classe.

Elementos da Figura 2: matriz de confusão com a diagonal destacada e totais por classe, acurácia geral, resumo da configuração que gerou o resultado e, como extra, precisão e revocação por classe.

9 Entregáveis

- **Código-fonte completo** compactado (ZIP).
- **README** com: linguagem e versão utilizadas, como compilar/executar, bibliotecas de apoio usadas (comprovando que nenhuma é de IA) e decisões de implementação (ex.: inicialização dos pesos, uso de bias, arredondamento da camada oculta).
- **Relatório curto de experimentos** (1 a 2 páginas): ao menos três treinos variando função de transferência, η ou nº de neurônios ocultos, com a curva de erro e a matriz de confusão de cada um e uma breve análise.
- **Apresentação e arguição:** demonstração da ferramenta funcionando e explicação do código por todos os integrantes.